Code Explanation

Delta Live Tables Code Explanation

This code is used to create a series of Delta Live Tables (DLT) that process customer and order data from CSV files. The tables are used to clean and transform the data, and to create aggregated views of customer spending.

Overview
The code defines a series of DLTs that read data from CSV files, clean and transform the data, and create aggregated views of customer spending. The tables are created using the `@dlt.table` decorator, and the data is processed using PySpark.

Step 1: Define Schemas for Customer and Order Data
This step defines the schemas for the customer and order data. The schemas are used to specify the structure of the data when it is read from the CSV files.

```
customer_schema = StructType([
    StructField("CustId", StringType(), True),
    StructField("Name", StringType(), True),
    StructField("EmailId", StringType(), True),
    StructField("Region", StringType(), True)
])

order_schema = StructType([
    StructField("OrderId", StringType(), True),
    StructField("ItemName", StringType(), True),
    StructField("PricePerUnit", DoubleType(), True),
    StructField("Qty", IntegerType(), True),
    StructField("Date", DateType(), True),
    StructField("CustId", StringType(), True)
])
```

Step 2: Read Source CSV Data into Bronze Tables
This step reads the customer and order data from CSV files into two bronze tables: `customer_bronze` and `order_bronze`.

```
@dlt.table(
    name="customer_bronze",
    comment="Raw customer data from CSV"
)
def customer_bronze():
    return (
        spark.read.format("csv")
        .option("header", "true")
        .schema(customer_schema)
        .load("/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/customerdata")
    )

@dlt.table(
    name="order_bronze",
    comment="Raw order data from CSV"
```

```
)
def order_bronze():
    return (
        spark.read.format("csv")
        .option("header", "true")
        .schema(order_schema)
        .load("/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/orderda
ta")
    )
```

Step 3: Clean Customer Data
This step cleans the customer data by removing duplicates and null values.

```
@dlt.table(
    name="customer_silver",
    comment="Cleaned customer data"
)
def customer_silver():
    return (
        dlt.read("customer_bronze")
        .dropDuplicates()
        .filter(
            (F.col("CustId").isNotNull()) &
            (F.col("Name").isNotNull()) &
            (F.col("EmailId").isNotNull()) &
            (F.col("Region").isNotNull())
        )
    )
```

Step 4: Clean Order Data and Add TotalAmount Column
This step cleans the order data by removing duplicates and null values, and adds a `TotalAmount` column.

```
@dlt.table(
    name="order_silver",
    comment="Cleaned order data with TotalAmount calculated"
)
def order_silver():
    return (
        dlt.read("order_bronze")
        .withColumn("TotalAmount", F.col("PricePerUnit") * F.col("Qt
y"))
        .dropDuplicates()
        .filter(
            (F.col("OrderId").isNotNull()) &
            (F.col("ItemName").isNotNull()) &
            (F.col("PricePerUnit").isNotNull()) &
            (F.col("Qty").isNotNull()) &
            (F.col("Date").isNotNull()) &
            (F.col("CustId").isNotNull())
        )
    )
```

Step 5: Create SCD Type 2 Ordersummary Table
This step creates an SCD Type 2 table called `ordersummary` that combines customer and order data.

```
@dlt.table(
    name="ordersummary",
    comment="SCD Type 2 table combining customer and order data",
    table_properties={
        "delta.autoOptimize.optimizeWrite": "true",
        "delta.autoOptimize.autoCompact": "true"
    },
    partition_cols=["Date"],
    schema="sdlc_wizard",
    catalog="gen_ai_poc_databrickscoe"
)
def ordersummary():
    current_customer = dlt.read("customer_silver")
    joined_data = (
        dlt.read("order_silver")
        .join(
            current_customer,
            on="CustId",
            how="inner"
        )
        .select(
            "CustId", "Name", "EmailId", "Region", "OrderId",
            "ItemName", "PricePerUnit", "Qty", "Date"
        )
    )
    result = (
        joined_data
        .withColumn("IsActive", F.lit(True))
        .withColumn("StartDate", F.current_timestamp())
        .withColumn("EndDate", F.lit(None).cast(TimestampType()))
    )
    return result
```

Step 6: Create Customer Aggregate Spend Table
This step creates a table called `customeraggregatespend` that aggregates customer spending by date.

```
@dlt.table(
    name="customeraggregatespend",
    comment="Aggregated customer spending by date",
    table_properties={
        "delta.autoOptimize.optimizeWrite": "true",
        "delta.autoOptimize.autoCompact": "true"
    },
    schema="sdlc_wizard",
    catalog="gen_ai_poc_databrickscoe"
)
def customeraggregatespend():
    return (
        dlt.read("order_silver")
        .join(
            dlt.read("customer_silver"),
```

```
            on="CustId",
            how="inner"
        )
        .groupBy("Name", "Date")
        .agg(F.sum("TotalAmount").alias("TotalAmount"))
    )
```